



Universidade Federal do Ceará
Centro de Tecnologia
Curso de Engenharia de Computação

Projeto 01 — Processamento Digital de Imagens

Ampliando e Reduzindo Imagens por Replicação de Pixels

Felipe Cordeiro de Sousa – 571941

Fortaleza, 03 de Outubro de 2025

Esse relatório tem, por finalidade, ilustrar como funciona a implementação da técnica de replicação de pixels em imagens digitais usando Python com as bibliotecas NumPy e OpenCV, conforme solicitado na disciplina de Processamento Digital de Imagens. Serão abordados a implementação dessa mesma técnica usando dois métodos, NumPy (otimizada) e tradicional (pouco otimizada), tanto para ampliação quanto para redução, como caso especial de interpolação de vizinho mais próximo, que se relaciona estreitamente com a sobreamostragem.

1 Discussão Técnica

Nesse relatório, abordaremos o conhecimento técnico necessário para ser possível reproduzir

Os resultados gerados pelo algoritmo de replicação de pixels ocasionando na ampliação ou redução de uma imagem digital está ilustrado no código 1 abaixo usando NumPy.

```
1 import cv2 as cv
2 import numpy as np
3
4 from typing import Literal
5
6 def ampl_red_pixels_numpy(type_resize:Literal["reducao", "ampliacao"],
7                             img:np.ndarray, factor:int=2):
8     if type_resize == "ampliacao":
9         new_img_gray = np.repeat(img, factor, axis=0)
10        new_img_gray = np.repeat(new_img_gray, factor, axis=1)
11    elif type_resize == "reducao":
12        new_img_gray = img_gray[::factor, ::factor]
13    return new_img_gray
```

Listing 1: Função para redimensionamento de imagem via replicação de pixels otimizada (usando NumPY).

```
1 import cv2 as cv
2 import numpy as np
3
4 from typing import Literal
5
6 def ampl_red_pixels(type_resize:Literal["reducao", "ampliacao"], img,
7                             factor:int=2):
8     height, width = img.shape
9
10    if type_resize == "ampliacao":
11        new_img_gray = np.zeros((factor*height, factor*width))
12        for h in range(height):
13            for w in range(width):
14                new_h = factor * h
15                new_w = factor * w
16                pixel_valor = img_gray[h, w]
17                new_img_gray[new_h:new_h+factor, new_w:new_w+factor] =
18                pixel_valor
19
20    elif type_resize == "reducao":
21        new_height, new_width = height // factor, width // factor
22        new_img_gray = np.zeros((new_height, new_width), dtype=img.
23        dtype)
24        for h in range(new_height):
25            for w in range(new_width):
26                orig_h, orig_w = h * factor, w * factor
27                new_img_gray[h, w] = img_gray[orig_h, orig_w]
28
29    else:
30        return img_gray
31
32    return new_img_gray
```

Listing 2: Função para redimensionamento de imagem via replicação de pixels

A principal diferença de desempenho entre a implementação tradicional (Código 2) e a otimizada (Código 1) reside na abordagem computacional adotada. O Código 2 utiliza laços explícitos (*for*) para percorrer a matriz da imagem pixel a pixel. Embora essa abordagem seja didática, ela é inerentemente lenta em Python devido à sobrecarga do interpretador em cada iteração.

Em contraste, a função do Código 1 emprega operações vetorizadas da biblioteca NumPy. Em vez de iterar em Python, ela delega as tarefas a rotinas de baixo nível, pré-compiladas em C, que operam sobre todo o array de uma só vez. Na ampliação, a função *np.repeat* realiza a replicação dos elementos de forma otimizada. Na redução, a técnica de fatiamento com passo (*img[:: factor, :: factor]*) executa uma subamostragem da imagem, selecionando os pixels desejados sem a necessidade de um laço. Essa diferença fundamental resulta em uma execução drasticamente mais rápida e em um código mais limpo e conciso.

Conforme descrito por Gonzalez e Woods, a interpolação por vizinho mais próximo envolve um mapeamento inverso de coordenadas da imagem de saída para a imagem de entrada [Gonzalez, 2009]. Para uma imagem de saída com coordenadas (x', y') e fatores de escala c_x e c_y , as coordenadas correspondentes na imagem de entrada, (x, y) , são dadas por:

$$x = \frac{x'}{c_x} \quad \text{e} \quad y = \frac{y'}{c_y} \quad (1)$$

A intensidade do novo pixel, $I'(x', y')$, é então atribuída com base na intensidade do pixel original I mais próximo da posição calculada. Isso é feito arredondando-se as coordenadas (x, y) para os inteiros mais próximos, resultando na seguinte equação:

$$I'(x', y') = I \left(\text{round} \left(\frac{x'}{c_x} \right), \text{round} \left(\frac{y'}{c_y} \right) \right) \quad (2)$$

onde a função *round()* arredonda seu argumento para o inteiro mais próximo. A Equação 2 define matematicamente a operação de replicação de pixels.

2 Resultados

Esta seção apresenta a análise visual dos resultados obtidos ao aplicar o algoritmo de redimensionamento por vizinho mais próximo, conforme a metodologia descrita por Gonzalez e Woods. O experimento consistiu em duas etapas sequenciais: primeiro, a imagem original foi reduzida por um fator de 10 através de subamostragem; em seguida, a imagem resultante, de baixíssima resolução, foi ampliada de volta às suas dimensões originais utilizando a técnica de replicação de pixels.

A Figura 1 exibe os quatro estágios do processo. A imagem do canto inferior esquerdo ("Imagem Reduzida") evidencia a drástica perda de detalhes após a subamostragem. Como este processo descarta 99 de cada 100 pixels, a maior parte da informação de alta frequência (bordas, texturas finas) é permanentemente eliminada.

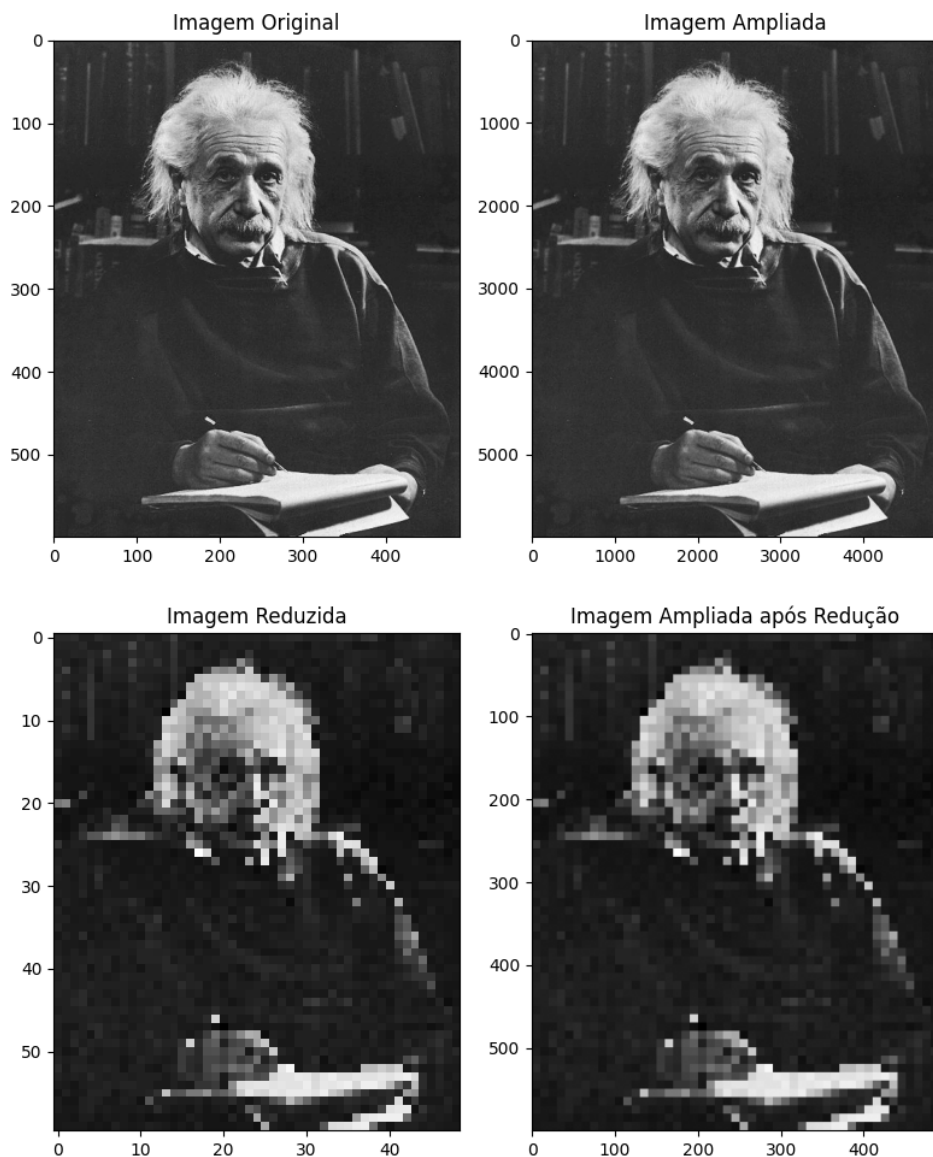


Figura 1: Análise comparativa das imagens geradas.

O resultado final é observado na imagem do canto inferior direito (“Imagem Ampliada após Redução”). Ao tentar reconstruir a imagem original a partir de uma base de dados tão degenerada, o algoritmo de ampliação não tem informação nova para trabalhar. Ele somente replica os poucos pixels restantes, criando os blocos quadrados que dominam a imagem. Este efeito de “pixelização” grosseira é uma manifestação severa do fenômeno de aliasing, que ocorre quando um sinal é amostrado a uma taxa insuficiente para capturar seus detalhes [Gonzalez, 2009].

3 Conclusão

Conclui-se, portanto, que a interpolação é um processo de estimacão, e não de criação. Uma vez que a informação é perdida na etapa de redução, não há como recuperá-la através de uma simples ampliação. O resultado final tem uma qualidade visual muito inferior

à original, demonstrando o caráter irreversível da perda de dados na subamostragem. A referência para a discussão teórica deste fenômeno é a obra de Gonzalez e Woods [Gonzalez, 2009], em especial o capítulo 4.

Referências

[Gonzalez, 2009] Gonzalez, R. C. (2009). *Digital image processing*. Pearson education india.